

LUMON — High-Level Design

Conversational Business Data Analyst — standalone application architecture

Purpose

LUMON provides a ChatGPT-style conversational interface for business data analysis. Authenticated users upload structured datasets, maintain conversation history, execute generated Python analysis locally through Pyodide, and receive verified natural-language explanations and interactive Plotly visualizations.

Technology Stack

Layer	Technology	Responsibility
Frontend	React 19 + TanStack Router/Start + TypeScript	Client routing, state, dashboard and settings
Styling	Tailwind CSS + Radix UI	Responsive interface, components and theme
Persistence/Auth	Supabase	Authentication, profiles, conversations, messages, dataset metadata and pinning
AI	Google Gemini API	Planning, explanation, profiling and chat-title generation
Data execution	Pyodide + pandas/numpy/scikit-learn	Browser-side computation against the active dataframe
Visualization	Plotly	Interactive chart rendering
Deployment	Vercel-compatible Node/Vite deployment	Hosting and production delivery

System Components

- Authentication and profile layer: Supabase Auth manages sessions; profiles store name, role and theme.
- Conversation layer: the sidebar and chat routes load conversations and messages belonging to the signed-in user.
- Dataset layer: an uploaded file is parsed in the browser, profiled, and associated with the active conversation. The dataframe itself remains in the browser runtime.
- AI orchestration layer: server functions send prompts to Google Gemini. The model selects chat mode or code mode and generates Python when computation is required.
- Execution layer: Pyodide executes the generated Python against df. The result, optional table and Plotly figure specification are returned to the UI.
- Explanation layer: executed output is sent back to Gemini. The model is instructed to use only numbers present in the execution output.
- Presentation layer: Markdown, tables, Plotly charts, follow-up chips, notebook export and dashboard pinning expose results to the user.

Primary Data Flow

1. User authenticates.
2. User creates or opens a conversation.
3. User uploads CSV/TSV/JSON/Excel.
4. Browser loads the dataframe and generates a structural profile.
5. Profile is persisted as dataset metadata and the active dataframe remains conversation-scoped in browser state.

- 6. User asks a natural-language question.
- 7. Server-side Gemini planner receives conversation history, role and dataset context.
- 8. If analysis is required, Gemini returns Python code.
- 9. Pyodide executes the code locally.
- 10. Execution output is passed to Gemini for a verified explanation.
- 11. UI renders text, tables, charts and follow-up actions; the assistant message is persisted.

Security Boundaries

- AI credentials remain server-side and are never placed in client-side source code.
- Supabase row-level security scopes persistent records to the authenticated user.
- The dataframe is processed locally in the browser rather than uploaded to the AI provider as raw file contents.
- Generated Python runs in the browser's WebAssembly/Pyodide environment rather than as arbitrary server-side code.
- Real production deployments must keep .env files and service credentials out of Git.

Current Scope and Explicit Limitation

- This standalone cleanup removes the app-builder-specific build configuration and AI gateway and replaces the AI gateway with a direct Google Gemini API call.
- Supabase remains the persistence/authentication backend in this cleaned baseline so that the existing application behavior is preserved.
- A full Supabase-to-Turso migration is a separate backend migration because Supabase currently supplies authentication plus database access; Turso alone is not a drop-in replacement for all of those services.
- Actual uploaded file bytes are not stored in the current database schema; dataset persistence currently stores metadata while the dataframe is held by the browser conversation state.